



Super Mario Suika: Clone Arcade in C#

Student: Mario Deusto Ríos

Vocational Training Cycle: Higher Level Application Development
Multiplatform (DAM)

Academic Year: 2025/2026

Tutor: Virginia Checa Calpena

INDEX

1. INTRODUCTION AND OBJECTIVES.....	3
1.1. Context and Motivation.....	3
1.2. General Objectives.....	3
1.3. Specific Objectives.....	3
2. BACKGROUND AND STATE OF THE ART.....	4
3. ANALYSIS.....	5
3.1. Detailed description of the solution and workflow.....	5
3.2. Functional Requirements.....	5
3.3. Non-Functional Requirements.....	5
3.4. Preliminary Data Model.....	5
4. DESIGN.....	6
4.1. Design of the Main Modules.....	6
4.2 Development Methodology.....	6
4.3. System Architecture.....	7
4.4. Use case diagrams.....	7
4.5. Class Diagrams.....	8
4.6. Interface Design.....	9
5. RESULTS AND IMPLEMENTATION.....	10
6. CONCLUSIONS.....	11
6.1. Conclusions of the work carried out.....	11
6.2. Future work.....	11
7. BIBLIOGRAPHY / WEBGRAPHY.....	12

1. INTRODUCTION AND OBJECTIVES

The technical report below details all the planning, development and project results, documenting the creation of a 2D video game from zero.

1.1. Context and Motivation

The development of physics-based video games with fusion mechanics has experienced exponential growth in the casual market. The project solves the challenge technician to program a custom physics engine and a system data persistence using only standard graphical tools Windows (Windows Forms), without resorting to pre-built external graphics engines such as Unity or Godot. The main motivation is to consolidate knowledge of Object-Oriented Programming (OOP), event handling, and management local files.

1.2. General Objectives

The overall goal of the system is to design, develop, and deploy a video game. A functional, interactive, and optimized arcade that allows users to interact with a real-time physics environment, guaranteeing the persistence of its records and offering a complete audiovisual experience.

1.3. Specific Objectives

- **Develop your own physics engine:** Implement gravity calculations, friction and circular collisions based on the Pythagorean Theorem.
- **Implement data persistence:** Design a read and write system in plain text files (.txt) to store a "Top 5" of historical scores.
- **Audiovisual management:** Integrate a looped background music system and effects sound reactive to game events.

Mario Deusto Ríos

1 DAM A – Intermodular Project

- **Interface design:** Develop an intuitive start menu and gameplay loop (Game Loop) stable at 60 frames per second.

2. BACKGROUND AND STATE OF THE ART

Currently, the physical puzzle subgenre has been popularized by titles like Suika Game. These systems are based on the accumulation of objects in a closed container, where contact between two identical entities generates a new entity of greater volume, requiring spatial management by the user. Technically, previous solutions usually depend on libraries of physical libraries like Box2D. This project does without such libraries to demonstrate the ability to calculate collisions and elastic impulses using mathematics fundamentals applied in C#.

3. ANALYSIS

3.1. Detailed description of the solution and workflow

The system features a welcome screen where the user can view the "Top 5 Historical Records". At the start of the game, the user controls a launcher horizontally at the top of the screen. The workflow consists of doing Click to drop "mushrooms" of different levels (sizes). The system processes internally, collisions: if two mushrooms of the same level touch, they merge, They score points and generate a sound; if a mushroom crosses the upper boundary line, the The game ends, evaluating whether the score obtained deserves to be saved in the local records file.

3.2. Functional Requirements

- **Launch Control:** The system must allow the user to move the projectile horizontally with the mouse and release it with a click.
- **Merging Mechanics:** The system must detect collisions and merge entities from the same level, increasing the score.
- **Records Management:** The system must read a local file on startup and overwrite it at the end of a game if a new maximum score is reached.
- **Testing Tools:** The system must include keyboard shortcuts (Ctrl + G) for force the "Game Over" state.

3.3. Non-Functional Requirements

- **Performance:** The main loop should run at intervals of 16 milliseconds to guarantee a visual refresh rate of 60 FPS.
- **Stability:** The system must avoid crashes when handling multiple collisions simultaneous through the use of controlled creation and destruction lists.

3.4. Preliminary Data Model

- **Config:** Static entity that centralizes physical constants (gravity, friction), screen sizes and the temporary record list.
- **GameData:** Entity that stores the current state of the game (score, mushrooms) active, completion status).
- **Mushroom:** Entity that represents each physical object (X/Y coordinates, level, radius, vector velocity).

4. DESIGN

4.1. Design of the Main Modules

- **Presentation Module (Partida.cs, Bienvenida.cs):** Manages the capture of keyboard/mouse events and graphics rendering using the Graphics object and tools like Pen.
- **Logic and Physics Module (GameService.cs):** Evaluates the behavior of the entities in each frame (separation of overlaps, bounces, boundaries of screen).
- **Persistence Module:** Manages I/O flow using System.IO for synchronize the static Config class with the physical storage on the hard drive.

4.2 Development Methodology

For the execution of this project, an iterative and incremental development methodology has been adopted, based on the principles of agile methodologies.

Since it is an individual project, this approach has made it possible to divide the technical complexity of the video game into short and manageable work cycles, guaranteeing the production of a functional and testable prototype at the end of each phase.

The development life cycle has been structured in the following main iterations:

- **1. Visual Analysis and Prototyping Phase:** Initial configuration of the work environment in Visual Studio, definition of the canvas dimensions (Config.cs), import of graphic assets (mushroom textures) and design of the basic graphic interface (Windows Forms forms).
- **2. Basic Physics Engine Phase:** Implementation of the game loop (Game Loop at 60 FPS) using timers. Development of fundamental physics: application of gravity vectors, horizontal friction, and control of screen boundaries (bounces off walls).
- **3. Logic and Collisions Phase:** Design and implementation of the core mathematical algorithm for detecting circular collisions. The resolution of elastic impacts (bounces between entities at different levels) was programmed.

and the merger mechanism (destruction and creation of new higher-level entities).

- **4. Rules Integration Phase (Gameplay):** Incorporation of the player controller (mouse movement and launch *cooldown*), point accumulation system and detection of the defeat condition (Game Over) when exceeding the Y limit coordinate.
- **5. Persistence and Audiovisual Polishing Phase:** Integration of the System.IO library for persistent writing/reading of the local record file.
Inclusion of sound feedback via SoundPlayer for background music and special blending effects, plus logical error resolution (control of multiple entity cloning and audio overlaps).

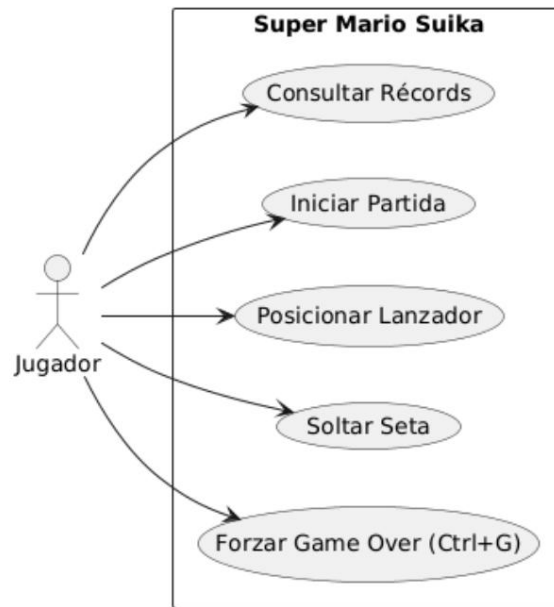
Throughout the entire process, priority has been given to the use of good Object-Oriented Programming (OOP) practices, applying continuous refactoring to ensure that the code complied with the single responsibility principle, strictly separating the visual presentation from the mathematical logic of the game.

4.3. System Architecture

An architecture based on the Separation design pattern has been used. The visual layer (Forms) does not perform physical calculations; it delegates these tasks to the GameService, injecting it with a data model (GameData) that acts as the sole source of truth. The selected language is C# over the .NET framework (Windows Forms), using its native SoundPlayer controller for the audio layer.

4.4. Use case diagrams

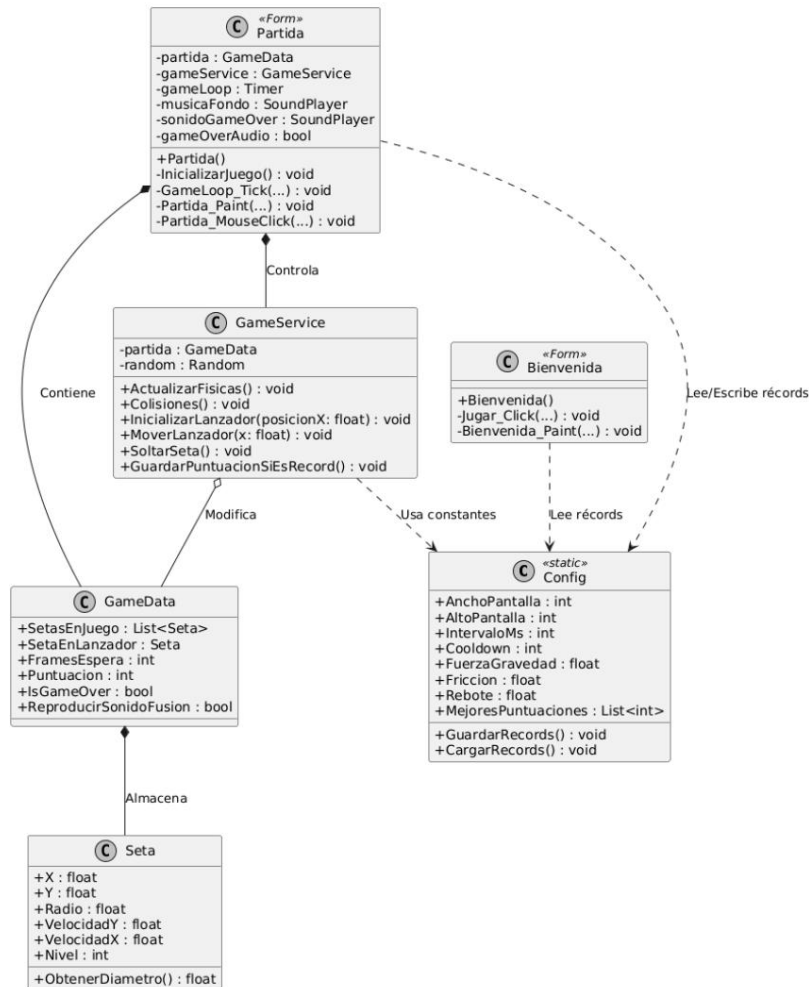
The system interacts with a single main actor (the Player), who triggers the various events of the game.



- **Use Case 1: View Records.** The player views the all-time Top 5 when opening the application.
- **Use Case 2: Start Game.** The player navigates from the main menu to playing canvas.
- **Use Case 3: Position Launcher.** The player moves the mouse to move the current mushroom horizontally.
- **Use Case 4: Drop Entity.** The player clicks so that gravity physics affects the mushroom and it falls.
- **Use Case 5: Force Game Over.** Using the developer shortcut (Ctrl + G), the user abruptly ends the game to test the save.

4.5. Class Diagrams

The project's object-oriented design is structured into classes with unique responsibilities and unidirectional dependencies to avoid coupling.



- **Config (Static Class):** Contains constant public attributes (GravityForce, Friction, ScreenWidth) and the static list BestScores.

Methods: SaveRecords(), LoadRecords().

- **Mushroom:** Basic entity. Attributes: X, Y, Radius, SpeedX, SpeedY, Level. Method: GetDiameter().
- **GameData:** State container. Attributes: Score, IsGameOver, MushroomsInGame (List of Mushroom objects), MushroomInLauncher (Mushroom object).
- **GameService:** Logic. Receives a GameData object via dependency injection in its constructor. Methods: UpdatePhysics(), Collisions(), DropMushroom().
- **Partition (Inherited from Form):** Graphical interface. Contains instances of GameService, GameData and SoundPlayer.

- **Welcome (Inherited from Form):** Initial graphical interface. Depends on Config to display records.

4.6. Interface Design

The graphical user interfaces (GUIs) have been designed under the principle of usability and immediate *feedback* , using GDI+ for real-time drawing.

- **Welcome Screen (Main Menu):** A static canvas that displays the project title and focuses attention on the score history.
It features a single action button ("Play") clearly visible to start the main flow.
- **Game Screen (HUD and Canvas):**
 - **Interaction area:** Canvas with black background of 500x800 pixels.
 - **HUD (Heads-Up Display):** Top label that updates the score in real time in the upper left corner.
 - **Danger Line:** White dotted line (Rendered using the Pen class) 150 pixels from the top margin that visually delimits the defeat condition (Game Over).
 - **Visual and auditory feedback:** When the defeat condition is activated, the interface blocks user input, overlays a pre-rendered "GAME OVER" sign, and triggers an audible alert.

5. RESULTS AND IMPLEMENTATION

The development concluded with a robust system capable of handling Autonomous complex calculations. As a relevant code snippet to demonstrate The system's logic highlights the implementation of collision avoidance to prevent errors. logical of "multiple cloning":

```
C# // Redundancy check for collisions (GameService.cs) bool areDeleted = mushroomsDelete.Contains(mushroomA)
|| mushroomsDelete.Contains(mushroomB);

if (areDeleted == false) {

    float distanceToSquare = (distanceX * distanceX) + (distanceY * distanceY);

    // Collision logic using the Pythagorean Theorem
}
```

Furthermore, the window closure insulation was successfully resolved by Native FormClosing event, ensuring that data is recorded to disk hard work happens safely and invisibly to the user regardless of the method closing.

6. CONCLUSIONS

6.1. Conclusions of the work carried out

The objectives set have been fully met. The project demonstrates the feasibility of creating complex and fluid interactive experiences without third-party engines. The physics system handles the transfer of energy and the collision of dozens of simultaneous elements at 60 FPS, and the Data persistence adds a high replay value to the system.

6.2. Future work

The modular design of the code allows for anticipating future improvements that have been discarded for this first phase, such as:

- **Preview System:** Implement a visual indicator (Next) that anticipates the player will be told the type of mushroom they will receive on the next throw.
- **Particle Effects:** Add a small animation or graphic explosion to the exact moment when two entities merge.

7. BIBLIOGRAPHY / WEBGRAPHY

- Microsoft. (2024). System.Drawing Namespace. Official .NET documentation.
<https://learn.microsoft.com/es-es/dotnet/api/system.drawing>
- Microsoft. (2024). SoundPlayer Class. Official .NET documentation.
<https://learn.microsoft.com/es-es/dotnet/api/system.media.soundplayer>